



تقرير وظيفة برمجة تفرعية

اختصاص ذكاء فئة 7

High-Performance E-Commerce Backend Engine

تقدمة الطلاب :

- جودي سعيد همام اسماعيل
- ناي نضال المقداد
- أحمد فيصل غريب
- حيدر أحمد عيد
- حيان ياسين صالح

High-Performance E-Commerce Backend Engine

3	: (Project Introduction)مقدمة المشروع
3	... (Concurrent Access & Data Integrity)الطلب الأول: حماية البيانات المشتركة من التضارب
8	 (Resource Management & Capacity Control)الطلب الثاني : إدارة الموارد الحاسوبية
11	 (Asynchronous Processing)الطلب الثالث: المعالجة غير المتزامنة
13	 (Batch Processing)الطلب الرابع : المعالجة الدفعية
18	 Load Balancing & Auto Scaling :الطلب الخامس

1. مقدمة المشروع (Project Introduction) :

في سوق التجارة الإلكترونية الحديث، تعد الكفاءة والسرعة والاعتمادية هي الركائز الأساسية لنجاح أي منصة. يهدف هذا المشروع إلى بناء **محرك خلفي عالي الأداء** قادر على معالجة آلاف الطلبات المتزامنة (Concurrent Requests) في نفس اللحظة دون انهيار النظام أو حدوث أخطاء في البيانات.

تم اختيار إطار العمل **Laravel (PHP)** لبناء هذا المحرك، وذلك بفضل بنيته التحتية المتطورة التي توفر أدوات قوية لإدارة الكاش، والعمليات المؤجلة (Queues)، والتعامل المتقدم مع قواعد البيانات. يركز المشروع بشكل أساسي على تحقيق **المتطلبات غير الوظيفية**

(Non-Functional Requirements - NFRs) مثل:

- **الموثوقية وسلامة البيانات (Data Integrity)** : ضمان عدم ضياع أو تضارب البيانات المالية والمخزون.
- **القابلية للتوسع (Scalability)** : قدرة النظام على التوسع مع زيادة عدد المستخدمين.
- **الزمن الاستجابي المنخفض (Low Latency)** : معالجة الطلبات بأسرع وقت ممكن.

الطلب الأول: حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity)

المشكلة (The Race Condition Problem)

عندما يحاول أكثر من مستخدم شراء نفس المنتج في نفس الأجزاء من الثانية، يقرأ كلا الطلبين نفس قيمة المخزون الحالي من قاعدة البيانات مثلاً و يقوم كلا الطلبين بطرح الكمية المطلوبة وحفظ النتيجة، مما يؤدي إلى تحديث خاطئ للمخزون وهو ما يعرف برمجياً بـ **Race Condition** ويسبب كارثة البيع الزائد (Overselling) .

الحلول المقترحة (Proposed Solutions) :

الحل الأول: القفل المتفائل (Optimistic Locking) :

يعتمد على فكرة أن التضارب نادر الحدوث. يتم إضافة عميل **version** أو **timestamp** لجداول المنتجات. عند التحديث، يتم التأكد من أن النسخة لم تتغير. إذا تغيرت، يفشل الطلب ويُطلب من المستخدم إعادة المحاولة.

- **المميزات** : أداء سريع لأنه لا يغلق قاعدة البيانات.

□ **العيوب:** سيئ جداً في حالات الضغط العالي والطلبات المتزامنة الكثيرة، لأن الكثير من الطلبات ستفشل وتطلب إعادة المحاولة.

الحل الثاني: القفل على مستوى قاعدة البيانات (Database-Level Locking (Pessimistic Locking :

يفترض هذا الحل أن التضارب سيحدث حتماً. عند قراءة المنتج، يتم حجز السطر في قاعدة البيانات

(Row Lock) باستخدام lockForUpdate(). أي طلب آخر يحاول قراءة أو تعديل نفس السطر سيتوقف و ينتظر (Blocking) حتى ينتهي الطلب الأول.(Commit/Rollback).

□ **المميزات:** يضمن سلامة البيانات بنسبة 100% على مستوى قاعدة البيانات مباشرة.

□ **العيوب:** قد يسبب بطء إذا طالت فترة القفل، لأنه يستهلك اتصالات قاعدة البيانات.(DB Connections)

الحل المطبق :

في هذا المشروع سنعتمد على Pessimistic Locking داخل Database Transaction كحل أساسي ومباشر داخل Laravel لضمان عدم حدوث Overselling نهائياً، والسبب في ذلك:

1. **اتساق البيانات المطلق (ACID Compliance) :** يضمن أن عملية التحقق من المخزون والتعديل تتم ككتلة واحدة لا تتجزأ.

2. **منع الـ Overselling تماماً:** يضع المستخدم B في حالة انتظار (Blocking) تلقائي حتى ينتهي المستخدم A من إنقاص المخزون، ليقرأ B القيمة الجديدة الصحيحة .

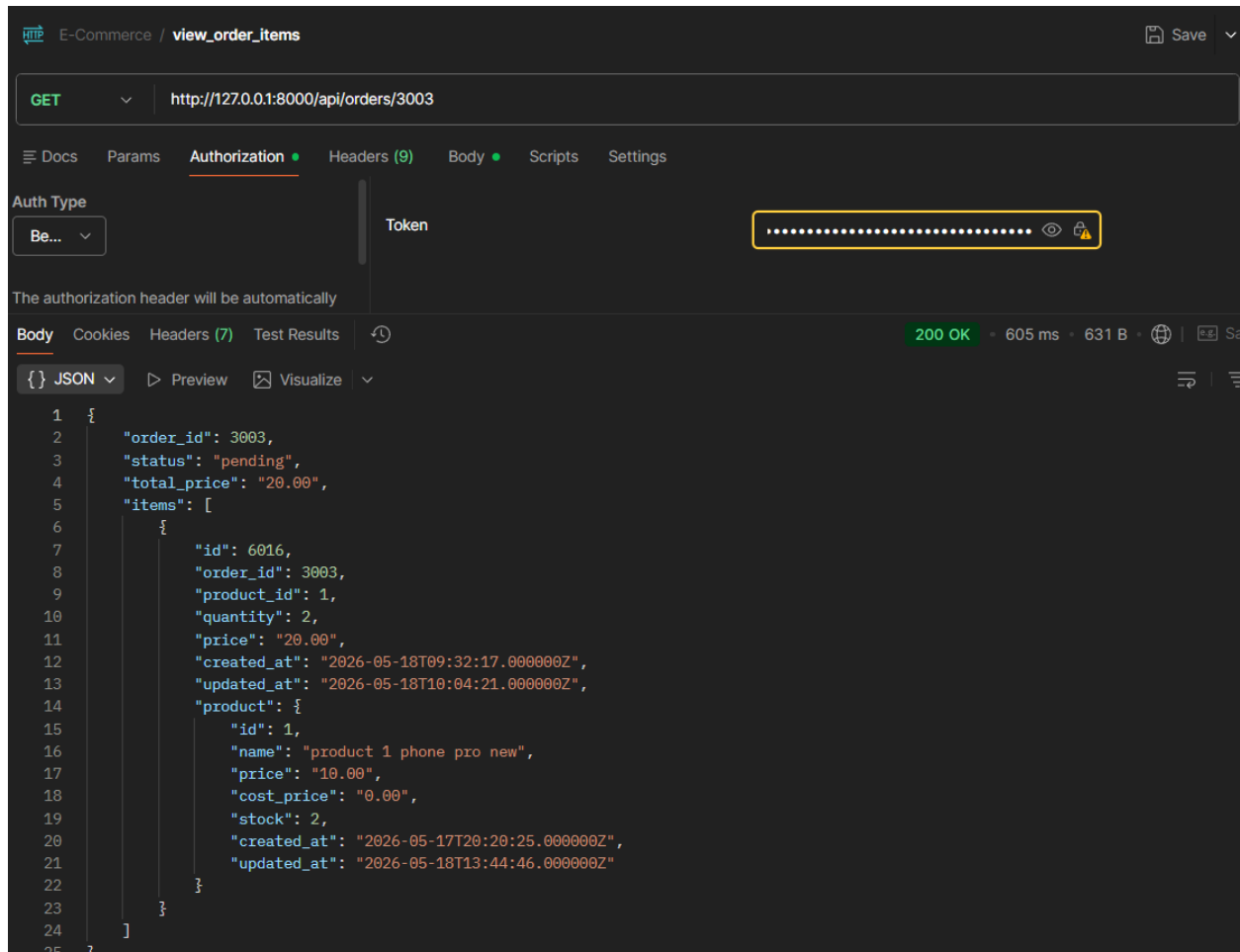
دليل اختبار التزامن والـ Race Condition (How to Test)

اختبار المشكلة :

لإثبات ثغرة الـ **Race Condition** في تابع (confirmOrderBroken)، استخدمنا Apache Server عبر بيئة Laragon لمعالجة الطلبات المتعددة بالتوازي (Multi-threading)، وأداة Apache JMeter لمحاكاة الهجوم الضغط المتزامن.

تم تجنب استخدام سيرفر PHP المدمج (artisan serve) لأنه أحادي المسار (Single-threaded) ويخفي مشكلة الـ Race Condition بتنفيذ الطلبات بالتوالي.

توضح هذه الصورة الطلب الذي نتعامل معه ، طلب لعنصرين من منتج واحد



المخزن للمنتج يحوي عنصرين فقط :

e_commerce.products: 20,053 rows total		Next	Show all	Sorting	Columns (7/7)	Filter
id	name	price	cost_price	stock	created_at	updated_at
1	product 1 phone pro new	10.0	0.0	2	2026-05-17 20:20:25	2026-05-18 10:09:22

إعدادات إرسال طلب POST المتزامن لـ Endpoint الخاص بتأكيد الطلب، مع تفعيل الـ Synchronizing Timer لضمان خروج الطلبات معاً (كل الطلبات لنفس المنتج وبالكمية نفسها) :

نلاحظ ان جميع الطلبات تم قبولها :

وهكذا تم التأكد من حدوث الشجرة ، تم قبول جميع الطلبات رغم ان المخزن غير كافي :

e_commerce.products: 20,053 rows total								Next	Show all	Sorting	Columns (7/7)	Filter
id	name	price	cost_price	stock	created_at	updated_at						
1	product 1 phone pro new	10.0	0.0	0	2026-05-17 20:20:25	2026-05-18 13:44:46						

إختبار حل المشكلة :

إعدادات إرسال الطلب بالتابع (confirmOrderFixed):

HTTP Request

Name: HTTP Request

Comments:

Basic Advanced

Web Server

Protocol [http]: http Server Name or IP: 127.0.0.1 Port Number:

HTTP Request

POST Path: /E-Commerce/public/api/orders/3003/confirm-fixed Content encoding:

☐ Redirect Automatically ☒ Follow Redirects ☒ Use KeepAlive ☐ Use multipart/form-data ☐ Browser-compatible headers

Parameters Body Data Files Upload

Send Parameters With the Request:

Name:	Value	URL Encode?	Content-Type	Include Equals?

نلاحظ انه تم قبول طلب واحد فقط أي تم مراعاة نفاذ المخزون :

View Results Tree

Name: View Results Tree

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes Configure

Search: Case sensitive Regular exp. Search Reset

Text

Sampler result Request Response data

HTTP Request

Thread Name: Thread Group 1-4

Sample Start: 2026-05-18 17:55:18 EET

Load time: 876

Connect Time: 3

Latency: 868

Size in bytes: 568

Sent bytes: 578

Headers size in bytes: 342

Body size in bytes: 226

Sample Count: 1

Error Count: 0

Data type ("text"|"bin"|""): text

Response code: 200

Response message: OK

HTTPSampleResult fields:

ContentType: application/json

DataEncoding: null

الطلب الثاني : إدارة الموارد الحاسوبية (Resource Management & Capacity Control)

المشكلة التي يعالجها هذا الجزء هي كيفية إدارة الموارد الحاسوبية للنظام عند وجود عدد كبير من الطلبات المتزامنة، بحيث لا يستهلك التطبيق المعالج والذاكرة وقاعدة البيانات بشكل مفرط، ولا يتم تقليل الموارد بشكل يؤدي إلى بطء الاستجابة أو فشل الطلبات. في نظام التجارة الإلكترونية تظهر هذه المشكلة بشكل واضح في العمليات المتكررة مثل البحث عن المنتجات وإضافة منتج إلى السلة، لأن هذه العمليات قد تُنفَّذ من عدد كبير من المستخدمين بنفس الوقت.

في **Laravel** لا يوجد تعامل مباشر وواضح مع مفهوم **Thread Pool** لذلك تم الاعتماد على تقنيات بديلة لإدارة الموارد والتحكم بالضغط. استخدمنا فكرة قريبة من **Semaphore / bounded access** من خلال تحديد عدد الطلبات المسموح بها على العمليات الحساسة باستخدام **Rate Limiting**، بحيث لا يُسمح بعدد غير محدود من الطلبات بالدخول إلى نفس الجزء من النظام في نفس الوقت. كما تم تحسين طريقة تنفيذ العمليات نفسها، فتم نقل البحث من ذاكرة التطبيق إلى قاعدة البيانات باستخدام استعلامات محددة و **limit**، وتم تحسين عملية الإضافة إلى السلة باستخدام تحديث آمن للمخزون بشكل **atomic** لمنع مشاكل التزامن وتقليل العمل غير الضروري على قاعدة البيانات.

عمليات التحسين التي قمنا بها :

1- تحسين عملية البحث عن المنتجات :

في الكود غير المحسن كان البحث يجلب كل المنتجات من قاعدة البيانات إلى ذاكرة **Laravel** ثم يقوم بالفترة داخل التطبيق. هذا الأسلوب يستهلك ذاكرة أكبر، ويزيد حجم البيانات، ويصبح بطيئاً عند وجود عدد كبير من المنتجات. أما في الكود المحسن، تم نقل عملية البحث إلى قاعدة البيانات نفسها حيث لا يتم تحميل جميع المنتجات إلى ذاكرة التطبيق، بل يتم تنفيذ شرط البحث داخل قاعدة البيانات باستخدام **limit** حتى لا يرجع النظام عدداً كبيراً من النتائج في كل طلب و تم تحديد الأعمدة المطلوبة فقط باستخدام **select** بدلاً من جلب كامل بيانات المنتج. بهذه الطريقة أصبح البحث أخف على الذاكرة والمعالج والشبكة.

2- تحسين عملية الإضافة إلى السلة :

في عملية الإضافة إلى السلة، المشكلة الأساسية هي أن عدة طلبات قد تصل بنفس الوقت وتحاول تعديل نفس المنتج أو نفس قيمة المخزون. لذلك تم استخدام أسلوب **atomic update** في الكود المحسن الفكرة التي تم التركيز هي تنفيذ التحقق من المخزون وإنقاظه ضمن عملية واحدة على مستوى قاعدة البيانات أي التحقق من المخزون والتعديل يحدثان كعملية واحدة، بدلاً من أن يقرأ التطبيق المخزون أولاً ثم يعدله لاحقاً. هذا يمنع حالات **race condition** ويقلل الضغط غير الضروري على قاعدة البيانات.

أيضاً تم استخدام **transaction** يضمن أن عملية الإضافة إلى السلة تتم كوحدة واحدة. فإذا حدث خطأ أثناء إضافة العنصر أو تحديث السعر، يتم التراجع عن العملية بدلاً من ترك البيانات بحالة غير صحيحة.

تم تحديد عدد ال **request** المتزامنة باستخدام **Rate Limiting** لمنع الضغط الزائد على النظام.

استخدمنا ثلاث أدوات أساسية للاختبار:

1- k6 : الهدف منه كان قياس أداء النظام قبل وبعد التحسين من ناحية زمن الاستجابة، نسبة فشل الطلبات، وعدد الطلبات التي يستطيع النظام معالجتها.

2- سكربت المراقبة monitor-v2.ps1 : هذا سكربت PowerShell استخدمناه لمراقبة استهلاك الموارد أثناء تشغيل اختبار k6 يسجل مع الزمن استهلاك المعالج CPU والذاكرة RAM للعمليات المرتبطة بالمشروع PHP و MySQL و Apache الهدف منه هو الحصول على بيانات رقمية عن استهلاك الموارد أثناء الضغط

3- سكربت الرسم plot_resources.py : هذا سكربت Python استخدمناه لتحويل ملف القياسات الناتج من monitor-v2.ps1 إلى مخطط بياني. الهدف منه هو عرض استهلاك CPU والذاكرة مع الزمن بشكل واضح، حتى نقدر نقارن بين حالة النظام قبل التحسين وبعده

الاختبار الذي قمنا به :

تم تنفيذ اختبار ضغط باستخدام k6 لمدة 60 ثانية وبمعدل 5 طلبات/ثانية، مع تخصيص 20 مستخدماً افتراضياً والسماح بالوصول حتى 80 مستخدماً عند الحاجة، بحيث يحاكي الاختبار حملاً مختلطاً يتكون من 70% طلبات بحث عن المنتجات و 30% طلبات إضافة منتج إلى السلة.

نوع الاختبار constant-arrival-rate ، يعني k6 يحاول أن يحافظ على 5 عمليات بالثانية بغض النظر عن بطء الاستجابة.

تعليمات تشغيل اختبار كود ال Broken :

تشغيل مراقبة الموارد وتسجيل CPU/RAM بملف CSV

```
powershell -ExecutionPolicy Bypass -File .\tools\monitor-v2.ps1 -Out broken-metrics.csv -Seconds 70
```

تشغيل اختبار الضغط بـ k6

```
k6 run -e RATE=5 .\tests\k6\mixed-broken-v3.js
```

تحويل ملف الـ CSV إلى مخطط بياني للموارد

```
python .\tools\plot_resources.py broken-metrics.csv --title "Before resource management" --out broken-resources.png
```

تعليمات تشغيل اختبار كود ال Fixed :

```
powershell -ExecutionPolicy Bypass -File .\tools\monitor-v2.ps1 -Out fixed-metrics.csv -Seconds 70
```

```
k6 run -e RATE=5 .\tests\k6\mixed-fixed-v3.js
```

```
python .\tools\plot_resources.py fixed-metrics.csv --title "After resource management" --out fixed-resources.png
```

نتائج الاختبار :

قبل التحسين :

1- النظام لم يستطع المحافظة على معدل الطلبات المطلوب، وظهرت طلبات فاشلة بسبب timeout

2- نسبة فشل الطلبات وصلت إلى 25.80%

3- وظهر dropped_iterations = 84، وهذا يدل على أن النظام لم يستطع معالجة كل الطلبات المطلوبة ضمن الزمن المحدد.

بعد التحسين :

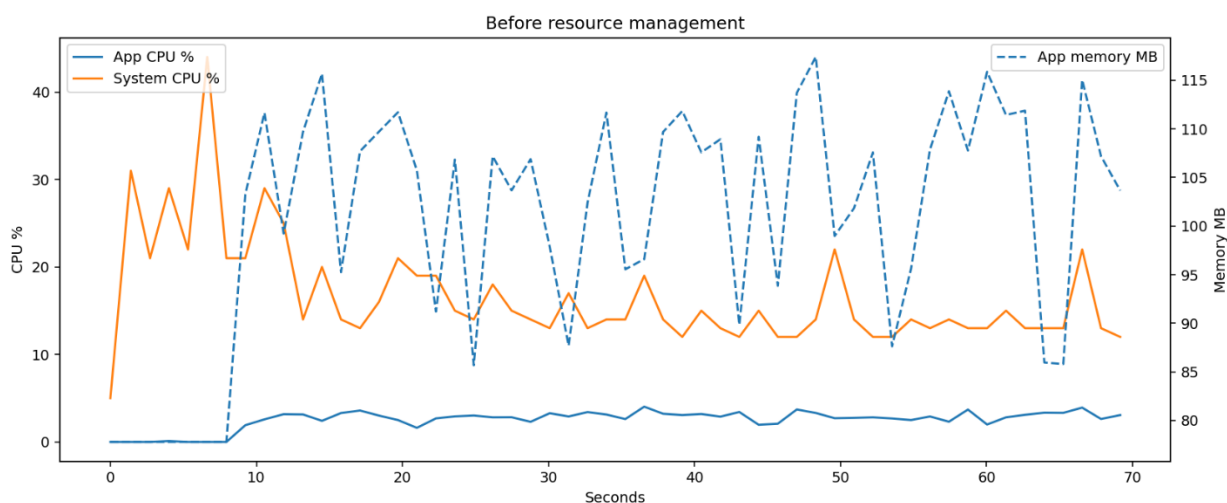
1- نسبة فشل الطلبات أصبحت 0%

2- وزمن الاستجابة p95 انخفض إلى 242ms

3- تم تنفيذ 300 طلب بنجاح

4- كما انخفض حجم البيانات المستقبلية إلى 420 KB فقط، ولم تظهر timeouts أو dropped iterations

ملاحظة حول استهلاك المعالج CPU وجدنا أن كل طلب HTTP في Laravel له كلفة ثابتة مثل تشغيل الـ middleware والـ routing والتعامل مع قاعدة البيانات وتجهيز الاستجابة. لذلك قد يظهر استهلاك CPU قريباً في الحالتين، لكن في الكود المحسن كان الـ cpu أكثر استقراراً .



جدول لمقارنة النتائج :

Metric	Before / Broken	After / Fixed
Failed requests	25.80%	0%
p95 response time	30s	242ms
Completed iterations	217	300
Dropped iterations	84	0
Received data	115 MB	420 KB
Request timeouts	Many timeouts	No timeouts
System behavior	Unstable under load	Stable under load

الطلب الثالث: المعالجة غير المتزامنة (Asynchronous Processing)

المشكلة التي يعالجها هذا الجزء

المشكلة الأساسية هي أن بعض العمليات التي تحدث بعد تأكيد الطلب مثل توليد الفاتورة وإرسال الإشعارات لا يحتاج المستخدم إلى انتظارها. في النظام غير المحسن، كانت هذه العمليات تنفذ داخل نفس دورة الطلب والاستجابة، مما يجعل المستخدم ينتظر +5 ثوانٍ قبل أن يتلقى أي رد. هذا يؤدي إلى بطء واضح في الاستجابة وضغط غير ضروري على الخادم تحت الحمل العالي.

عمليات التحسين التي قمنا بها

1 - النسخة (Broken) معالجة متزامنة وبلوكينج:

في الكود غير المحسن، كانت جميع العمليات تنفذ بشكل متسلسل داخل نفس الطلب:

- ☐ حفظ الطلب
- ☐ توليد الفاتورة (3 ثوانٍ)
- ☐ إرسال الإشعار (2 ثوانٍ)

□ ثم فقط يُرسل الرد للمستخدم

المستخدم ينتظر كل هذه العمليات قبل أن يرى أي استجابة. إذا فشلت أي خطوة، يفشل الطلب كله. تحت الحمل العالي، كل PHP worker يبقى محجوزاً لمدة +5 ثوانٍ لكل طلب، مما يعني أن عدداً قليلاً جداً من الطلبات يمكن معالجتها بالتوازي.

2 - النسخة المحسنة (Fixed) معالجة غير متزامنة عبر Queue :

في الكود المحسن، تم تقسيم العمليات إلى:

المسار الحرج (Critical Path) ينفذ فوراً قبل إرسال الرد:

□ تأكيد الطلب وتحديث الحالة إلى paid

المهام الخلفية (Background Tasks) تُرسل إلى Queue ولا ينتظرها المستخدم:

□ توليد الفاتورة → ProcessInvoiceJob

□ إرسال الإشعار → SendNotificationJob

يتلقى المستخدم الرد فوراً بعد تأكيد الطلب، بينما يعالج Workers هذه المهام في الخلفية بشكل مستقل.

3 - آلية إعادة المحاولة والـ Dead Letter Queue :

تم تكوين كل Job بـ 3 محاولات مع Exponential Backoff:

□ المحاولة الأولى: انتظار 5 ثوانٍ

□ المحاولة الثانية: انتظار 10 ثانية

□ المحاولة الثالثة: انتظار 15 ثانية

بعد استنفاد جميع المحاولات، ينتقل الـ Job إلى جدول الـ failed_jobs الذي يمثل الـ Dead Letter Queue، حيث يمكن مراجعته يدوياً أو إعادة تشغيله لاحقاً.

4 - عزل الـ Queues :

تم تخصيص queue منفصل لكل نوع من المهام:

□ invoices queue → لمهام توليد الفواتير

□ notifications queue → لمهام الإشعارات

هذا يعني أن ضغطاً على الإشعارات لن يؤخر توليد الفواتير والعكس صحيح. يتم تشغيل worker مستقل لكل queue .

أداة الاختبار المستخدمة

تم استخدام Apache JMeter لاختبار الأداء، حيث تم إرسال 50 طلباً متزامناً على كلا النسختين وقياس زمن الاستجابة والـ Throughput ونسبة الأخطاء.

نتائج الاختبار

قبل التحسين Synchronous Broken :

النظام يعالج كل طلب بشكل كامل داخل دورة الطلب، مما يجعل كل PHP worker محجوزاً لمدة 5+ ثوانٍ. تحت حمل 50 مستخدماً متزامناً، تراكمت الطلبات وارتفع زمن الاستجابة بشكل كبير.

بعد التحسين Asynchronous Fixed :

النظام يُنهي المسار الحرج فوراً ويُرسل المهام الثقيلة إلى Queue. PHP worker يتحرر بسرعة لاستقبال الطلب التالي، مما يرفع ال-Throughput ويخفض زمن الاستجابة بشكل ملحوظ.

جدول مقارنة النتائج

Metric	Before / Broken	After / Fixed
Average response time	7,546 ms	3,076 ms
Min response time	5,858 ms	1,017 ms
Max response time	10,095 ms	5,247 ms
Error %	0.00%	2.80%
Throughput	3.5 req/sec	5.7 req/sec
Processing mode	Synchronous	Asynchronous
Background jobs	None	Invoice + Notification

الطلب الرابع : المعالجة الدفعية (Batch Processing)

في أنظمة التجارة الإلكترونية ذات الحجم الكبير، تُعدّ عملية إنشاء تقارير المبيعات اليومية من أكثر العمليات استهلاكاً للموارد أن كنا نريد معالجتها بالطرق التقليدية (أخذ السجلات كاملة دفعة واحدة ثم معالجتها) و ستواجهنا مشكلتين :

1 - Memory Exhaustion : يؤدي تحميل عشرات الآلاف من السجلات إلى الذاكرة العشوائية دفعة واحدة إلى استهلاك مفرط للموارد، مما قد يتسبب في انهيار السيرفر وتوقف الخدمة بالكامل.

2 - Request Timeout : تفرض بيئات الويب حداً أقصى لزمان معالجة طلب HTTP عند تجاوز هذا الحد بسبب ضخامة البيانات، ينقطع الاتصال تلقائياً قبل إتمام العملية، مما يجعل التقرير غير قابل للاسترداد.

لمعالجة هاتين المشكلتين تم اعتماد **Batch Processing** .

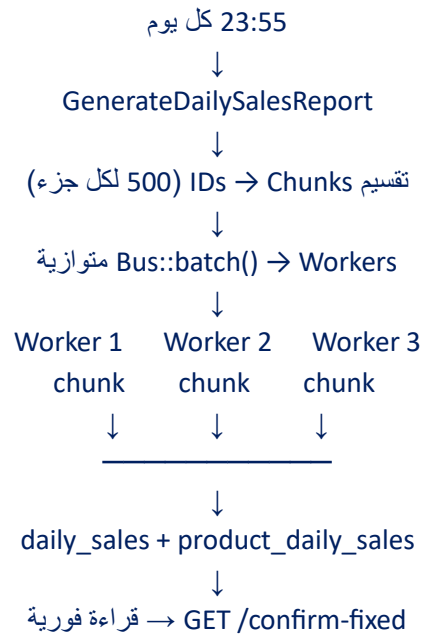
بنية النظام :

يعتمد النظام على بنية مكونة من ثلاث طبقات تعمل بشكل متسلسل ومتواز:

1 - Chunk Generation : يتم تشغيل GenerateDailySalesReport تلقائياً كل يوم عند الساعة 23:55 عبر Laravel Scheduler تقوم هذه Job بسحب جميع معرفات الطلبات الدفوعة لليوم و تقسيمها على دفعات بحجم 500 سجل لكل جزء إرسالها دفعةً واحدة إلى Bus::batch() ثم تقسيمها إلى أجزاء متوازية Workers (مبدأياً علينا فتح تيرمينال لكل ووركر نريده).

2 - Processing : ProcessSalesReportChunk تستلم كل جزء على حدة وتعالجه باستقلالية تامة تقوم بتحميل الطلبات مع عناصرها ومنتجاتها، إجراء الحسابات اللازمة (الأرباح، صافي الربح، عدد الطلبات ...) ثم حفظ النتائج في جدول daily_sales و product_daily_sales باستخدام نمط Upsert لضمان سلامة البيانات تحت التوازي.

3 - Query : عند طلب التقرير عبر fixedReport النظام يقرأ فقط من الجداول الجاهزة ويعيد النتيجة فوراً لا يجري أي حساب و الجداول لن تكون كبيرة ستكون إحصائيات تتعلق بعدد المنتجات.



ماذا قمنا :

أولاً القرارات المتعلقة بالتوازي والبنية:

لماذا Queue وليس تنفيذ مباشر؟ Laravel لا تدعم تشغيل العمليات بشكل متوازٍ مباشرة داخل HTTP request ، لذلك تم اعتماد نظام ال Queue كوسيط يستلم ال Jobs ويوزعها على Workers تعمل باستقلالية تامة في الخلفية.

لماذا 3 طبقات وليس Job واحدة؟ الفصل بين طبقة التوليد (Generation) وطبقة المعالجة (Processing) كان قراراً مقصوداً. عملية إنشاء الـ Workers وتسليمها الـ Chunks تستغرق وقتاً وكون العملية خلفية تعمل في وقت محدد (23:55)، فإن فصل المسؤوليات يجعل كل طبقة مركزة على مهمة واحدة فقط، مما يسهل قراءة الكود و إصلاحه ويزيد الأداء.

لماذا Bus::batch() وليس dispatch عادي؟ Bus::dispatch() يرسل Job واحدة في كل مرة يعني الـ Workers تعالج Chunks بشكل تسلسلي. أما Bus::batch() فيرسل جميع الـ Jobs دفعةً واحدة، مما يسمح لعدة Workers بمعالجة عدة Chunks في نفس الوقت وهذا هو جوهر التوازي في النظام. إضافةً إلى ذلك، يوفر batch تتبعاً تلقائياً لتقدم الـ Jobs وإدارة منظمة للأخطاء. اي كل ما ينتهي Worker سيستلم الـ chunk جديد .

ثانياً حجم الـ Chunk:

لم يكن اختيار حجم الـ Chunk عشوائياً بل تم اختياره للموازنة بين :

- عدد صغير جداً مثلاً 50 مما يُثقل النظام ويزيد عدد اتصالات قاعدة البيانات
- عدد كبير جداً مثلاً 5000 يعيدنا لمشكلة استنزاف الذاكرة

لذلك تم اختيار 500 كقيمة وسطى تُبقي استهلاك الذاكرة لكل الـ Jobs محدوداً مع إبقاء عدد الـ Worker ويجب الملاحظة بأن هذا الرقم يتفاوت من نظام لآخر حسب البيانات و الموارد التي يمتلكها كل نظام.

ثالثاً Upsert وليس Insert عادي؟

بما أن عدة Workers تعمل بالتوازي، فمن الممكن أن يكون نفس المنتج موجوداً في أكثر من Chunk في نفس الوقت. مثلاً:

□ Worker 1 حسب: منتج 5 باع 300 وحدة

□ Worker 2 حسب: منتج 5 باع 200 وحدة

لو استخدمنا insert أو update عادي، سيكتب أحد الـ Workers فوق الآخر والنتيجة ستكون 300 أو 200 بدلاً من 500 — أي بيانات خاطئة.

الحل كان استخدام upsert مع DB::raw على هذا الشكل:

```
'quantity_sold' => DB::raw('quantity_sold + VALUES(quantity_sold)')
```

هذا يعني: بدلاً من استبدال القيمة أضف عليها والعملية atomic على مستوى قاعدة البيانات أي لا يمكن لـ Worker آخر أن يتدخل في منتصفها النتيجة: $500 = 200 + 300$

وإذا كان السجل غير موجود أصلاً، يقوم بـ Insert تلقائياً. وإذا كان موجوداً، يقوم بـ Update تراكمي كل هذا في عملية واحدة آمنة.

رابعاً 23:55 وليس منتصف الليل؟

لو شغلنا الـ Job عند 00:00 أو 00:05، فإن today() في Laravel سيعطي اليوم الجديد يعني سنعالج طلبات يوم فارغ، وبيانات أمس تضيق للأبد. تشغيلها عند 23:55 يضمن أن today() لا يزال يشير لليوم الصحيح.

المقارنة بين brokenReport و fixedReport :

الجانب	brokenReport	fixedReport
آلية العمل	يحمل كل الطلبات في الذاكرة ويحسب مباشرة	يقرأ من جداول جاهزة مسبقاً
الذاكرة	استهلاك مفرط يتناسب مع عدد الطلبات	ثابت ومحدود بغض النظر عن حجم البيانات
زمن الاستجابة	يزداد مع ازدياد الطلبات	فوري دائماً
تحت الضغط	يتعطل	يعمل بثبات
Timeout	محتمل جداً	مستحيل
الحسابات	تتم لحظة الطلب	تمت مسبقاً في الخلفية

نتائج اختبار الأداء JMeter :

تم اختبار النظام باستخدام Apache JMeter على 3000 طلب موزع عليها 6000 عنصر مع 100 مستخدم متزامن، وكانت النتائج كالتالي:

metrics	brokenReport	fixedReport (processing)	fixedReport (query)
Average	42,788 ms	26,368 ms	17,569 ms
Min	1,056 ms	623 ms	529 ms
Max	82,903 ms	52,879 ms	34,224 ms
Throughput	1.1/sec	1.6/sec	2.3/sec
Error %	0%	0%	0%

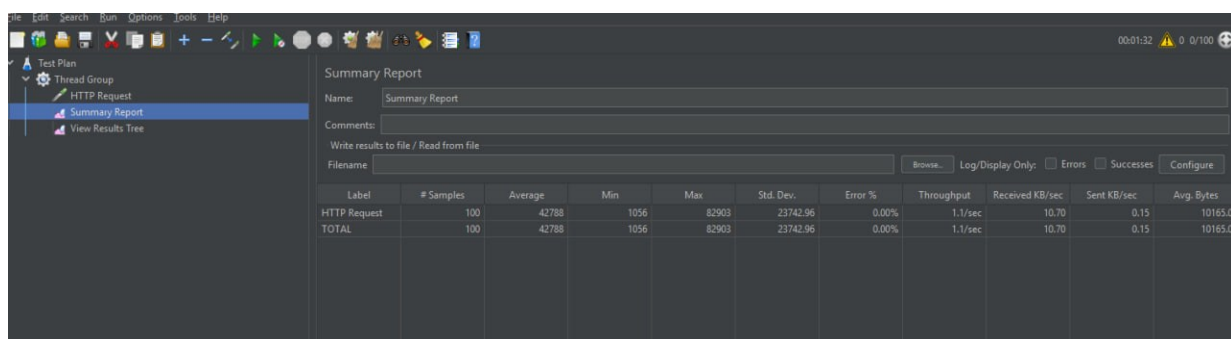
ملاحظة: لتشغيل الـ broken يكفي <http://localhost:8000/api/sales/report/confirm-broken>

أما لتشغيل الـ fixed فيوجد خطوتين generate مع فتح تيرمنالز و workers و هنا سيبدأ بتوليد و الحفظ داخل الداتا ثم confirm-fixed

<http://localhost:8000/api/sales/report/generate>

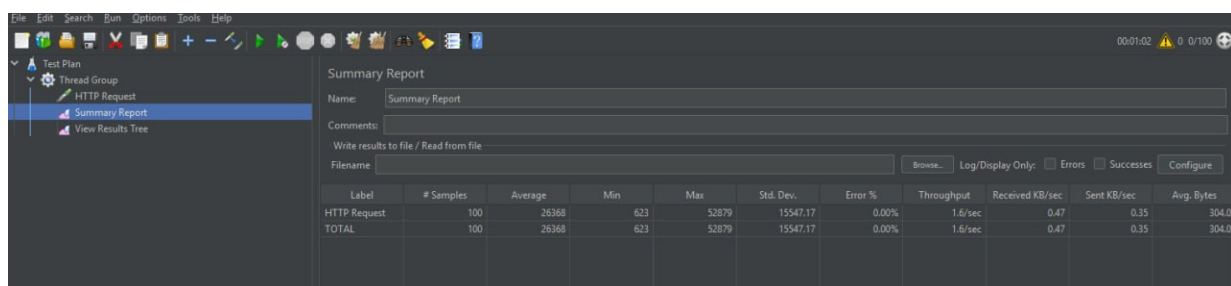
<http://localhost:8000/api/sales/report/confirm-fixed>

تحليل النتائج:



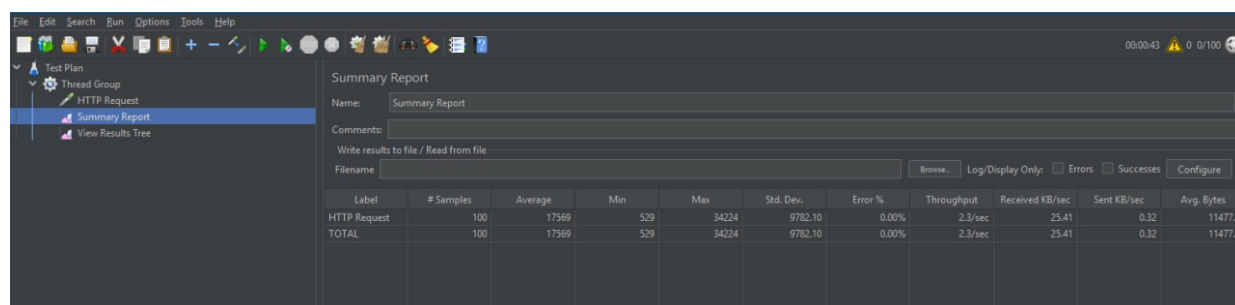
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	100	42788	1056	82903	23742.96	0.00%	1.1/sec	10.70	0.15	10165.0
TOTAL	100	42788	1056	82903	23742.96	0.00%	1.1/sec	10.70	0.15	10165.0

brokenReport (42,788ms average) الأبطأ لأنه يحتمل كل السجلات في الذاكرة ويجري الحسابات كاملة داخل الـ HTTP request مباشرة كل مستخدم يسبب ضغطاً كاملاً على السيرفر وقاعدة البيانات في نفس الوقت.



Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	100	26368	623	52879	15547.17	0.00%	1.6/sec	0.47	0.35	304.0
TOTAL	100	26368	623	52879	15547.17	0.00%	1.6/sec	0.47	0.35	304.0

fixedReport (26,368ms average) أسرع من brokenReport لأن الحسابات تجري في الخلفية بشكل موزع، لكنه لا يزال يستغرق وقتاً لأن الـ Workers لا تزال تكتب في قاعدة البيانات.(عمل الخلفية).



Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	100	17569	529	34224	9782.10	0.00%	2.3/sec	25.41	0.32	11477.9
TOTAL	100	17569	529	34224	9782.10	0.00%	2.3/sec	25.41	0.32	11477.9

fixedReport (17,569ms average) الأسرع لأن السيرفر لا يجري أي حساب يقرأ فقط من جداول جاهزة ويعيد النتيجة فوراً.

الفرق في الـ Throughput واضح: من 1.1/sec في brokenReport إلى 2.3/sec في fixedReport أي أن النظام الجديد يخدم ضعف عدد المستخدمين في نفس الوقت.

ملاحظة: تم الاختبار في بيئة محلية ذات موارد محدودة. في بيئة الإنتاج مع workers متعددة وقاعدة بيانات منفصلة، يكون الفارق في الأداء أكبر بكثير لصالح fixedReport

العمل على التوازي :

<pre> 2026-05-17 08:28:54 App\Jobs\GenerateDailyS h-performance-ecommerce\vendor RUNNING 2026-05-17 08:28:56 App\Jobs\GenerateDailyS alesReport 1s DONE 2026-05-17 08:28:56 App\Jobs\ProcessSalesRe portChunk RUNNING 2026-05-17 08:29:14 App\Jobs\ProcessSalesRe portChunk 17s DONE 2026-05-17 08:29:14 App\Jobs\ProcessSalesRe portChunk </pre>	<pre> 2026-05-17 08:28:57 App\Jobs\ProcessSalesRe portChunk RUNNING 2026-05-17 08:29:17 App\Jobs\ProcessSalesRe portChunk 20s DONE </pre>	<pre> 2026-05-17 08:28:57 App\Jobs\ProcessSalesRe portChunk RUNNING 2026-05-17 08:29:17 App\Jobs\ProcessSalesRe portChunk 20s DONE </pre>
---	---	---

تعليمة الجدولة :

```
Schedule::job(new GenerateDailySalesReport)->dailyAt('23:55');
```

الطلب الخامس : Load Balancing & Auto Scaling

1. نظرة عامة على المشكلة

في النظام الأساسي (قبل التحسين)، كانت جميع الطلبات تتجه إلى خادم واحد. هذا يعني نقطة فشل واحدة، واستهلاك غير محدود للموارد، وانتهاء تام تحت الضغط العالي.

لذا فنحن نهذف الى:

- جعل التطبيق لا يعتمد على حالة محلية (Stateless) حتى يمكن توزيعه على خوادم متعددة
- توزيع الطلبات على عدة خوادم عبر Load Balancer كـ Nginx
- تقسيم اتصالات قاعدة البيانات إلى قراءة وكتابة
- بناء نظام مراقبة وتوسع تلقائي (Auto Scaling)
- إثبات التحسين عبر اختبارات أداء حقيقية باستخدام JMeter

2. جعل التطبيق عديم الحالة (Stateless)

إذا خزن الخادم بيانات الجلسة في ذاكرته المحلية، فإن الطلب التالي من نفس المستخدم يجب أن يصل إلى نفس الخادم وهذا يلغي فائدة التوزيع. بنقل الجلسات إلى Redis (مخزن مشترك خارجي)، يصبح أي خادم قادراً على معالجة أي طلب.

تم تغيير قيمتين في ملف .env:

```
SESSION_DRIVER=redis
```

```
CACHE_STORE=redis
```

لماذا Redis تحديداً؟

- مخزن في الذاكرة (In-Memory) أي سرعة استجابة فائقة
- مشترك بين جميع الخوادم أي لا يوجد تبعية للخادم
- يستخدمه النظام أيضاً للـ Queue أي توحيد البنية التحتية

ملاحظة: نظام المصادقة مبني على JWT، وهو بطبيعته عديم الحالة لذا الخادم لا يحتاج لتخزين أي شيء للتحقق من الهوية، بل يتحقق من التوقيع الرقمي للـ Token.

3. التبديل من SQLite إلى MySQL

SQLite قاعدة بيانات مبنية على ملف واحد. في بيئة متعددة الخوادم، لا يمكن لأكثر من خادم الوصول لنفس الملف في وقت واحد بشكل موثوق. لذلك كان التبديل إلى MySQL (خادم قاعدة بيانات حقيقي).

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=ecommerce
DB_USERNAME=root
DB_PASSWORD=
```

4. تقسيم اتصالات قاعدة البيانات (Read/Write Split)

في التطبيقات الحقيقية، حوالي 80% من الاستعلامات هي قراءة (SELECT) لذا بتوجيه القراءة إلى خادم نسخة (Replica) والكتابة إلى الخادم الرئيسي (Primary) يتوزع الحمل ولا تصبح قاعدة البيانات عنق الزجاجة.

التعديل في config/database.php

```
'read' => ['host' => [env('DB_READ_HOST', '127.0.0.1')]],
'write' => ['host' => [env('DB_WRITE_HOST', '127.0.0.1')]],
'sticky' => true,
```

أهمية خيار sticky:

بدون sticky، بعد عملية كتابة، قد تذهب استعلامات القراءة إلى النسخة وتعود بيانات قديمة (تأخر النسخ المتماثل). مع sticky، تظل القراءة على نفس خادم الكتابة طوال نفس الطلب.

في .env. وضمن البيئة المحلية للتطوير

كلا المتغيرين DB_READ_HOST و DB_WRITE_HOST يشيران إلى 127.0.0.1 لمحاكاة النمط. في الإنتاج، يكفي تغيير DB_READ_HOST إلى عنوان IP الخادم النسخة.

5. إعداد Nginx كموازن للأحمال (Load Balancer)

Nginx هنا لا يقوم بتشغيل PHP بل يعمل ك Reverse Proxy يستقبل جميع الطلبات على المنفذ 8080 ويوزعها على خوادم Laravel المتعددة.

مسار الطلب:

المستخدم 8001 → Laravel:8001 → Nginx:8080 → أو 8002 أو 8003

خوارزمية التوزيع Least Connections :

تم اختيار least_conn بدلاً من round_robin لأن طلبات التجارة الإلكترونية تتباين كثيراً في وقت المعالجة (تصفح المنتجات سريع، بينما عملية الدفع والتحقق من المخزون أبطأ بكثير). Least Connections تضمن توجيه كل طلب جديد للخادم الأقل انشغالاً.

ملف الإعداد nginx.conf

```
worker_processes 1;

events { worker_connections 1024; }

http {
    error_log "C:/nginx/logs/error.log";
    access_log "C:/nginx/logs/access.log";

    upstream laravel_servers {
        least_conn;
        server 127.0.0.1:8001;
        server 127.0.0.1:8002;
        server 127.0.0.1:8003;
    }

    server {
        listen 8080;
```

```

location /nginx_status {
    stub_status on;
    access_log off;
    allow 127.0.0.1;
    deny all;
}

location / {
    proxy_pass http://laravel_servers;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
}
}

```

شرح كل جزء بالاعدادات

- `upstream laravel_servers`: يُعرّف مجموعة الخوادم الخلفية
- `least_conn`: خوارزمية التوزيع الخادم الأقل اتصالاً يستقبل الطلب التالي
- `listen 8080 Nginx`: يستمع على هذا المنفذ
- `proxy_pass`: يُحيل الطلب إلى مجموعة الخوادم
- `proxy_set_header`: يُمرر معلومات العميل الحقيقية للخادم الخلفي
- `location /nginx_status`: يكشف إحصائيات الاتصالات المباشرة يستخدمها Auto Scaler

الفحص الصحي السلبي (Passive Health Check)

Nginx يراقب صحة الخوادم بشكل سلبي تلقائياً: إذا فشل خادم في الاستجابة، يتوقف Nginx عن توجيه الطلبات إليه ويوزعها على الباقين. تم إثبات هذا عملياً بإيقاف خادمين من ثلاثة — النظام استمر في العمل بنسبة خطأ 0%.

7. نقطة الفحص الصحي /api/health/

```

Route::get('/health', function () {
    return response()->json([
        'status' => 'ok',
        'server' => gethostname(),
        'timestamp' => now(),
    ], 200);
});

```

الغرض

- تمكين Auto Scaler من التحقق من جاهزية الخادم الجديد قبل توجيه الطلبات إليه
- يُظهر اسم الخادم لتأكيد أن Nginx يوزع الطلبات فعلاً بين الخوادم المختلفة
- يُستخدم لإثبات الفحص الصحي السلبي بقتل خادم ومراقبة استمرار الاستجابة

8. نظام التوسع التلقائي (Auto Scaler)

Auto Scaler هو سكربت PHP يعمل في الخلفية ويراقب الحمل على الخوادم كل ثانية، ويتخذ قرار التوسع أو التقليل كل 5 ثوانٍ بحيث يقرأ Auto Scaler إحصائيات الاتصالات المباشرة من Nginx عبر:

http://127.0.0.1:8080/nginx_status

يُحلل الاستجابة باستخراج قيمة Active connections ثم يقسمها على عدد الخوادم للحصول على متوسط الحمل لكل خادم.

منطق قرار التوسع

- إذا كان متوسط الاتصالات لكل خادم < 25 → تشغيل خادم جديد (Scale Up)
- إذا كان متوسط الاتصالات لكل خادم ≥ 10 → إيقاف خادم زائد (Scale Down)
- خلاف ذلك → لا إجراء، عرض الحالة فقط

آلية التوسع

عند Scale Up:

- تشغيل خادم Laravel جديد على منفذ جديد باستخدام proc_open
- إضافة المنفذ الجديد إلى nginx.conf
- إعادة تحميل Nginx بأمر: `nginx.exe -p C:\nginx -s reload`

عند Scale Down:

- إيجاد PID العملية على المنفذ عبر netstat
- إنهاء العملية باستخدام taskkill
- حذف المنفذ من nginx.conf وإعادة تحميل Nginx

9. المراحل الثلاث للاختبار

المرحلة الأولى: بدون Auto Scaler (3 خوادم ثابتة، Nginx معطل التحديث)

في هذه المرحلة، Auto Scaler كان يُشغّل خوادم جديدة لكن Nginx لم يكن يوجه إليها الطلبات بسبب خطأ أمر الـ reload. الخوادم الأصلية الثلاثة كانت تتحمل العبء وحدها.

التشغيل الثالث	التشغيل الثاني	التشغيل الأول	المقياس
—	—	2000	عدد العينات
—	—	42.25%	نسبة الأخطاء
—	—	17,866ms	متوسط وقت الاستجابة
—	—	2ms	أقل وقت استجابة
—	—	37,787ms	أعلى وقت استجابة
—	—	20.9 طلب/ثانية	الإنتاجية

المرحلة الثانية: Auto Scaler مع حد أقصى 3 خوادم ديناميكية (6 إجمالاً)

بعد إصلاح أمر nginx reload، أصبح Auto Scaler يُشغّل خوادم جديدة ويوجه الطلبات إليها فعلاً. الحد الأقصى كان 6 خوادم (3 أساسية + 3 ديناميكية).

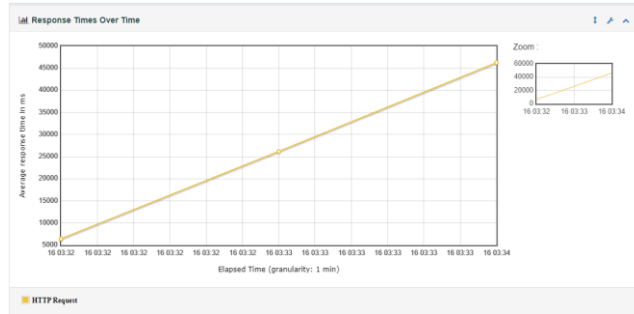
التشغيل الثالث	التشغيل الثاني	التشغيل الأول	المقياس
—	2000	—	عدد العينات
—	36.75%	—	نسبة الأخطاء
—	9,366ms	—	متوسط وقت الاستجابة
—	2ms	—	أقل وقت استجابة
—	23,741ms	—	أعلى وقت استجابة
—	27.1 طلب/ثانية	—	الإنتاجية

المرحلة الثالثة: Auto Scaler بخوادم لانهائية (range 8004–8099)

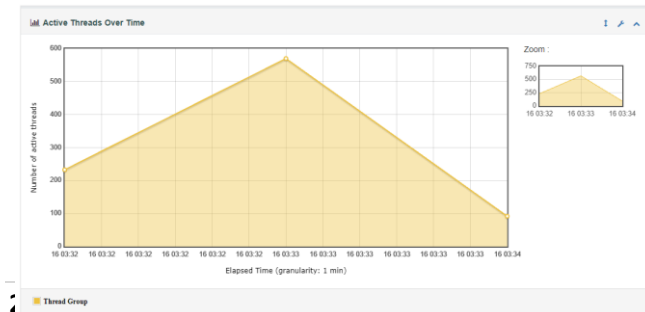
بتوسيع نطاق المنافذ الديناميكية من 3 إلى 96 خادماً ممكناً ((range(8004, 8099)، أصبح النظام يُضيف خوادم بلا حد أقصى عملي حتى يتعامل مع الحمل الكامل.

التشغيل الثالث	التشغيل الثاني	التشغيل الأول	المقياس
2000	—	—	عدد العينات
0.00% ✓	—	—	نسبة الأخطاء
21,434ms	—	—	متوسط وقت الاستجابة
271ms	—	—	أقل وقت استجابة
48,569ms	—	—	أعلى وقت استجابة
18.9 طلب/ثانية	—	—	الإنتاجية

ملاحظة: ارتفاع متوسط وقت الاستجابة في المرحلة الثالثة مقارنة بالثانية طبيعي لان النظام الآن يُعالج 100% من الطلبات بدلاً من رفض بعضها بسرعة. الاستجابة البطيئة الناجحة أفضل من الاستجابة السريعة الفاشلة.



نلاحظ ازدياد مدة معالجة الطلب مع مرور الزمن وذلك لان عملية اعداد خادم جديد تتطلب وقتاً (لكن بفضل ذلك أصبحت جميع الطلبات ناجحة حتى مع بعض التأخير)



نلاحظ وصول عدد نيا سب المعالجة لقمة في منتصف وقت المعالجة (حينها كان عدد الطلبات اكبر ما يمكن) وبعد معالجة الضغط تم إيقاف بعض الخوادم وبالتالي قل عدد

9. مقارنة ملخصة

الجانب	قبل التحسين	بعد التحسين
الجلسات	ملف محلي / قاعدة بيانات	Redis مشترك
قاعدة البيانات	SQLite (ملف)	MySQL (خادم)
توزيع الحمل	خادم واحد	Nginx + 3 خوادم
اتصالات DB	اتصال واحد	قراءة / كتابة منفصلان
التوسع	يدوي	Auto Scaler تلقائي
نسبة الأخطاء تحت الضغط	42.25%	0.00%